

LSTM-UT and Recurrent-Depth Transformers on Cellular Automata

Aras Kavuncu

University of Southampton, Electronics and Computer Science

Supervisor: Muhammad Burhan Hafez

September 2026

Abstract

Recurrent-depth Transformers apply shared computation repeatedly, but differ in how they retain information across steps. We compare a Block Universal Transformer (BUT), which carries only its current hidden state; CoTFormer, which also retains an expanding attention cache; and a new LSTM Universal Transformer (LSTM-UT) with bounded gated memory. On Rule 30 cellular automata, BUT extrapolates to unseen recurrent depths more reliably than CoTFormer, although its accuracy eventually degrades. State and cache interventions show that CoTFormer’s failure depends on their interaction: correcting the current state can temporarily restore accuracy, while retained history can undermine that correction. In a delayed-recall task, BUT also outperforms CoTFormer despite lacking direct access to past states; CoTFormer does not reliably select the requested cached representation. LSTM-UT improves both depth extrapolation and delayed recall over these baselines. The results support bounded gated memory as an effective inductive bias for repeated computation and later retrieval in these tasks.

Keywords: Machine Learning, Recurrent-Depth Transformers, Universal Transformers, Gated Memory, Cellular Automata, Depth Extrapolation

1 Introduction

Transformers conventionally tie computational depth to a fixed stack of parameter-distinct layers. Universal and looped Transformers instead repeatedly apply shared Transformer computation to an evolving latent state, separating the number of computational steps from the number of learned parameters [1], [2]. This makes it possible, at least architecturally, to allocate additional computation at inference time by executing more recurrent iterations than were used for a particular training example. Recent work has consequently investigated whether recurrent-depth Transformers can learn multi-step computations and generalise to greater reasoning depths by iterating for longer [2], [3].

Parameter sharing makes additional iterations well-defined, but does not guarantee that the learned computation remains useful when repeatedly composed. A recurrent model may fit every supervised repeat depth while learning internal dynamics that converge to an attractor, oscillate, or otherwise fail beyond the training range. Recurrent-depth extrapolation therefore depends not only on whether the architecture can execute additional computation, but on whether optimisation produces a stable and reusable latent transition.

The amount of computation performed and the information retained during that computation are also distinct resources. A model may execute many recurrent steps while carrying forward only its latest latent state, potentially overwriting information produced during earlier steps. Alternatively, it may maintain a persistent memory or expose earlier intermediate representations directly. This distinction has recently appeared in continuous latent reasoning. Coconut recursively feeds a continuous hidden representation back into a language model as the input to the next reasoning step [4], while more recent work argues that this repeated replacement can create a “concept bottleneck” and augments Coconut with a gated persistent stream intended to preserve information across reasoning passes [5]. Although these models differ from recurrent-depth Transformers, they raise the same architectural question: when is the latest latent state sufficient, and when must earlier computation remain separately accessible?

This paper compares three recurrent-depth Transformer architectures with contrasting memory structures: the Block Universal Transformer (BUT), which carries forward only its latest hidden sequence; CoTFormer [6], which additionally retains an expanding attention cache of earlier recurrent computation; and a Long Short-Term Memory Universal Transformer (LSTM-UT), introduced here as a bounded gated alternative. These architectures respectively represent bounded hidden-state recurrence, recurrence with an expanding latent history, and recurrence with bounded gated memory.

The empirical investigation proceeds in four stages. First, BUT and CoTFormer are compared on Rule 30 cellular automata to examine recurrent-depth extrapolation. Second, interventions on CoTFormer’s recurrent state and cache probe how the two components contribute to its loss of accuracy at greater depths. Third, a delayed-recall benchmark tests whether earlier intermediate states can be recovered after further computation, while attention diagnostics and cache interventions examine whether CoTFormer uses its retained history as query-addressable memory. These findings motivate the fourth stage: the introduction of LSTM-UT, which is evaluated on both Rule 30 extrapolation and delayed recall using the same benchmark definitions and aligned protocols.

The paper therefore asks whether a shared transition remains reliable beyond its supervised recurrent depth, whether direct access to earlier representations supports dependable retrieval, and whether bounded gated memory provides a more effective alternative. LSTM-UT is developed as an architectural response to the limitations identified in the initial comparison and is evaluated against both original architectures.

This study deliberately isolates these architectural properties rather than constructing a complete world model or reinforcement-learning agent. Rule 30 and delayed recall remove perception, stochastic environmental dynamics, reward learning, and action selection, allowing recurrent transition stability and latent memory to be examined under controlled conditions. The results therefore support claims about recurrent-depth extrapolation, access to intermediate computation, and bounded gated memory within these synthetic tasks; they do not establish that the same behaviours will transfer directly to realistic reasoning, planning, or world-modelling problems.

2 Background and Related Work

2.1 World Models as Learned Recurrent Dynamics

A world model repeatedly applies learned dynamics to predict future states. Each prediction becomes the input to the next step, so long rollouts depend on both a reliable transition and a representation that preserves information needed later.

One approach compresses the trajectory into a bounded recurrent state. Dreamer uses gated deterministic state alongside stochastic state to predict future observations and rewards [7], [8]. MuZero and TD-MPC2 likewise learn compact latent dynamics, retaining information useful for their prediction and control objectives rather than reconstructing every past observation [9], [10]. The memory footprint stays fixed as the rollout grows, but access to an earlier state depends on what the current representation has preserved.

Another approach keeps a directly accessible history. Transformer world models such as IRIS and Genie attend to sequences of earlier visual tokens while predicting future tokens [11], [12]. This exposes more of the trajectory to later computation, but the context grows with the rollout until it is truncated or compressed. Retaining history also does not guarantee that a model will select the relevant earlier representation when queried.

This state–history trade-off motivates the comparisons in this paper. We test whether a shared transition remains useful at unseen depths, whether access to earlier recurrent representations supports delayed recall, and whether bounded gated memory can preserve information without a growing history.

2.2 Universal and Looped Transformers

Here, Universal Transformer refers broadly to Transformer computation whose parameters are shared across recurrent depth; the repeated component may be a single layer or a short stack. A common encode–loop–decode layout places this shared block between non-recurrent input and output transformations. Thus, a k -layer block applied for L repeats contains k distinct recurrent layers but performs kL recurrent layer applications [2]; parameter count, recurrent depth, and total computation must therefore be distinguished. The motivation for recurrence is strongest when a task is naturally expressed as repeated application of an algorithmic step. Looped Transformers have been shown to learn iterative in-context learning algorithms with substantially fewer distinct parameters than parameter-untied baselines [13]. Related work on recurrent networks demonstrates an easy-to-hard form of extrapolation in which models trained on simpler instances improve on harder instances when allowed additional iterations at evaluation [14]. Saunshi et al. extend this perspective to addition, p -hop induction, synthetic mathematical reasoning, and language-model evaluation, finding that multi-layer looped models can approach parameter-untied models with the same effective depth [2]. More recent recurrent-depth work similarly studies whether additional inference-time loops support compositional and depth generalisation [3].

Weight sharing makes such extrapolation possible in principle, because the learned transition remains well-defined when applied more times. It does not guarantee that the transition learned by optimisation is stable or algorithmic outside the supervised range. A model can fit outputs at the trained depths while learning latent dynamics that converge to an attractor, oscillate, or otherwise fail when composed further. Training across several recurrent depths provides constraints on more compositions of the shared function, but whether this produces extrapolation remains an empirical question.

Recent work has approached recurrent-depth stability through fixed-point convergence. STARS encourages looped language models to approach asymptotically stable fixed points by regularising an estimate of the recurrent Jacobian’s spectral radius across sampled recurrent depths [15]. Fixed-Point Reasoners similarly treat convergence of the latent state as an adaptive halting criterion: the model continues iterating until its recurrent residual becomes sufficiently small [16]. In these formulations, each input may induce a different fixed point representing its completed solution. Harder instances can require more iterations to reach that point, after which further applications of the recurrent block should leave the representation approximately unchanged.

The objective studied in this paper is importantly different. Fixed-point convergence is appropriate when recurrence performs iterative refinement toward a terminal answer. In a learned transition model, however, each recurrent application should produce the next distinct state of an evolving process. For Rule 30, the desired computation is not convergence of $H^{(r)}$ to a stationary representation, but continued transition consistency. The ideal is therefore indefinite recurrent-depth extrapolation: the learned block should remain a valid reusable transition under arbitrarily many applications, rather than merely remain bounded or converge. A fixed point can constitute successful termination in the former setting but a spurious attractor in the latter.

Ouro demonstrates that looped computation can also be incorporated into large-scale language-model pretraining rather than being restricted to small synthetic tasks. Its LoopLM family performs iterative latent computation and learns depth allocation through an entropy-regularised objective [17]. This supports recurrent depth as a practical modelling direction, although performance on language benchmarks does not by itself establish that a recurrent transition behaves like a stationary environmental dynamics model.

Recent work distinguishes recurrent computation from persistent mutable memory, showing that additional iterations do not expand the working-memory capacity of compressed latent loops [18]. Although this limitation does not directly apply to full sequence-state models such as the BUT studied here, it motivates treating recurrent depth and preserved intermediate state as separate computational resources.

The BUT and CoTFormer occupy different points within this distinction. Both recurrently update a full sequence of token states, so neither is a compressed loop with only a small number of persistent latent slots. The BUT carries only the latest sequence state across depth, whereas CoTFormer additionally exposes keys and values saved from earlier recurrent states. The following subsection describes this architectural difference precisely. It then becomes possible to ask separately whether the shared transition extrapolates and whether retaining its earlier intermediate representations is beneficial.

2.3 The CoTFormer

CoTFormer is a recurrent-depth Transformer that retains representations from earlier applications of a shared block [6]. Unlike textual chain-of-thought, each repeat updates the full sequence of T hidden tokens rather than appending a new token. A Block Universal Transformer (BUT) carries only the latest hidden sequence $X^{(r)}$ into the next repeat. CoTFormer carries that sequence as well, but also stores the attention keys and values produced at every repeat. Later queries can therefore attend directly to earlier recurrent representations instead of relying on the current hidden state to preserve them.

We call this storage a CoT cache to distinguish it from an autoregressive KV cache, which stores past sequence positions during token generation. The CoT cache stores keys and values for the *same* sequence positions at successive recurrent depths within one forward pass. After repeat r , each block’s cache has rT repeat–token positions; with batch size B , H heads, and head width d_h , its key and value tensors each have shape $B \times H \times rT \times d_h$. Queries from the current T positions attend across those rT cached positions. The cache remains in the differentiable training graph and is cleared after the forward pass.

CoTFormer thus exposes an expanding latent history, whereas BUT must compress any useful history into its bounded current sequence. The attention projections do not depend on r , so both models can run for more repeats than they saw in training. Whether their learned transitions and memory use remain effective at those depths is an empirical question.

3 Rule 30 Cellular Automata: A Test of Depth Extrapolation

We selected the one-dimensional cellular automaton Rule 30 as the time-dependent transition task. A state at time t is a binary row

$$X^{(t)} = (x_0^{(t)}, \dots, x_{N-1}^{(t)}), \quad x_i^{(t)} \in \{0, 1\}.$$

We used periodic boundary conditions, so indices wrap around the ends of the row. One Rule 30 update is

$$x_i^{(t+1)} = x_{i-1}^{(t)} \oplus (x_i^{(t)} \vee x_{i+1}^{(t)}),$$

where $\oplus$ and $\vee$ denote XOR and OR. Equivalently, the eight possible left–centre–right neighbourhoods are mapped as follows:

(x_{i-1}, x_i, x_{i+1})	111	110	101	100	011	010	001	000
$x_i^{(t+1)}$	0	0	0	1	1	1	1	0

Initial rows were sampled independently from a Bernoulli distribution with probability 0.5 for each cell. Targets at greater depths were generated by repeatedly applying the exact transition above. The same model parameters were supervised at multiple update depths, with the number of shared-block repeats matched to the number of Rule 30 updates. The task therefore asks one recurrent computation to produce a valid state after any configured number of updates rather than learning only one fixed terminal prediction.

3.1 Rule-30 experimental setup

Causal attention prevents a cell from seeing the right neighbour required by Rule 30. Under the Bernoulli(0.5) initial-state distribution and periodic boundaries, the resulting optimal mean cell accuracy is 75%; both BUT and CoTFormer reached this ceiling. We therefore used bidirectional attention in all principal experiments, providing the complete row and predicting every output cell simultaneously.

Both models used a GPT-2-based Transformer ([19]) with a binary vocabulary, width 64, four attention heads, a width-256 feed-forward sublayer, rotary positional encoding, and no dropout. A single shared Transformer block was applied r times, with no separate input or output Transformer blocks (the 0/1/0 configuration). A preliminary 1/1/1 configuration performed worse and was not used in the principal comparisons. BUT carried only the latest hidden sequence between repeats; CoTFormer also retained keys and values from earlier repeats. A shared output projection produced two class scores per cell. We trained with cellwise cross-entropy and decoded all cells in parallel. The models therefore differed in access to recurrent history, rather than in the number of shared blocks or output procedure.

Training For each initial row $X^{(0)}$, an update-repeat pair $a:r$ specified a target $X^{(a)}$ obtained by applying the exact Rule 30 transition a times and a model run of r repeats. The principal experiments used diagonal pairs $a = r$, so each repeat corresponded to one automaton update. Only the final output was supervised; no clean intermediate states were supplied. Configured pairs were presented in round-robin order, one depth per optimiser step, with all depths present from the start of training.

We used three supervision ranges: 1:1–4:4 (R_4), 1:1–6:6 (R_6), and 1:1–12:12 (R_{12}). The R_4 and R_6 runs drew from the same pool of 100,000 initial rows and trained for 5,000 and 7,500 optimiser steps, respectively. The larger-budget R_{12} runs used 500,000 initial rows and 24,000 steps. All rows contained 64 cells sampled independently from Bernoulli(0.5). Validation and final-test sets each contained 4,096 independently generated rows. Random seeds controlled model initialisation and data generation; unless stated otherwise, a reported seed denotes the model-initialisation seed. Comparisons across model seeds held the data seeds and other scientific settings fixed.

Evaluation and checkpoint selection We measured cell accuracy and exact-row accuracy at trained and unseen depths. To diagnose failure, we also examined class balance and MCC, the target horizon best matched by each decoded repeat, agreement between adjacent decoded states and one exact Rule 30 update, and hidden-state similarity across repeats. These diagnostics distinguish inaccurate predictions, temporal stalling, and latent-state convergence; decoded transition agreement alone does not establish a correct trajectory.

Checkpoints were selected on validation data before final-test evaluation. The in-distribution selector maximised exact-row accuracy at the largest trained depth, using cell accuracy and then negative loss to break ties. The exploratory extrapolation selector maximised cell accuracy at unseen depth 7, 9, or 15 for R_4 , R_6 , or R_{12} , respectively. A stricter extrapolation selector used the same objective but required at least 0.99 cell accuracy and 0.95 exact-row accuracy at every trained depth. Results across model seeds report the number of seeds and their mean, sample standard deviation, minimum, and maximum.

3.2 Rule 30 Results

3.2.1 BUT exhibits more stable depth extrapolation than CoTFormer

At checkpoints selected using the unconstrained extrapolation criterion, BUT exhibited substantially more stable depth extrapolation than CoTFormer. In the R_4 experiments, BUT achieved near-perfect accuracy at the greatest supervised depth and retained high accuracy at unseen depths. The corresponding CoTFormer

checkpoints were already less accurate at the supervised depth and deteriorated rapidly as the number of inference-time repeats increased. Across the evaluated training regimes and model seeds, BUT remained accurate to greater recurrent depths than CoTFormer.

Exploratory runs supervised only through depth 2 did not extrapolate reliably. Extending supervision to greater depths improved extrapolation, consistent with the possibility that observing more compositions of the shared transition constrains its recurrent dynamics more strongly.

Table 1 summarises the standardised R_4 comparison. Depth 4 is supervised during training, whereas depths 6 and 9 measure extrapolation. Each row represents one architecture and hyperparameter configuration; cell accuracy is aggregated over the listed model seeds.

Table 1. Cell accuracy for models trained on R_4 . Values are the mean and sample standard deviation (SD) across the listed model seeds. Checkpoints were selected using the unconstrained extrapolation criterion. One representative hyperparameter configuration is shown.

Architecture	Learning rate	Weight decay	Gradient clip	Model seeds	4:4 (trained)		6:6		9:9	
					Mean	SD	Mean	SD	Mean	SD
BUT	10^{-3}	0.15	0.9	0, 1, 2, 3	0.9997	0.0006	0.9974	0.0051	0.9296	0.1330
CoTFormer	10^{-3}	0.15	0.9	1, 2, 3	0.9377	0.0599	0.6081	0.1849	0.5004	0.0012

Supervising through depth 6 exposes the recurrent transition to two additional compositions during training. Consistent with the R_4 results, the resulting CoTFormer models generalised farther than their R_4 counterparts, but their accuracy still deteriorated more rapidly than that of BUT as inference depth increased (Table 2).

Table 2. Cell accuracy for models trained on R_6 , using checkpoints selected by the unconstrained extrapolation criterion. Values are the mean and sample standard deviation (SD) across model seeds.

Architecture	Learning rate	Weight decay	Gradient clip	Model seeds	6:6 (trained)		9:9		12:12	
					Mean	SD	Mean	SD	Mean	SD
BUT	10^{-3}	0.1	0.9	0, 1, 2	1.0000	0.0000	0.9999	0.0001	0.9810	0.0297
CoTFormer	10^{-3}	0.1	0.9	0, 1, 2	0.9997	0.0003	0.8940	0.1234	0.5031	0.0062

3.2.2 BUT eventually collapses under excessive recurrence

BUT’s superior extrapolation relative to CoTFormer does not imply indefinite recurrent stability. In every evaluated BUT run, accuracy eventually deteriorated when the model was applied for sufficiently many repeats, although the depth at which this occurred varied across model seeds and training configurations. Most runs remained accurate well beyond their supervised range, in some cases for approximately twice the maximum training depth, before collapsing.

The observed failure cannot be explained solely as the accumulation of small errors from otherwise correct recurrent transitions. Early prediction errors were often concentrated near the spatial boundary. This pattern is consistent with a possible mismatch between RoPE’s linear relative-position geometry and the periodic boundary conditions of Rule 30, although the present experiments do not establish RoPE as the cause. At greater depths, however, decoded-recurrence accuracy also deteriorated substantially. Successive decoded states therefore ceased to approximate valid Rule 30 transitions, rather than merely drifting away from the exact trajectory through the accumulation of individually small transition errors.

The latent dynamics changed concurrently. Consecutive hidden states became increasingly similar, while the decoded outputs became progressively biased toward the all-ones state. Table 3 illustrates this behaviour for one representative R_4 BUT run. As diagonal accuracy approaches chance level, hidden-state

cosine similarity approaches one and decoded-recurrence accuracy falls sharply. Figure 1 shows the corresponding increase in similarity across recurrent depth.

Taken together, these observations are consistent with attractor-like convergence of the recurrent dynamics. They do not by themselves establish the existence or geometry of a particular latent attractor, because high cosine similarity can arise for other reasons. Nevertheless, the joint increase in hidden-state similarity, saturation of the decoded outputs, and loss of decoded transition consistency resembles the depth-dependent degradation diagnosed in recent analyses of looped Transformer dynamics [15].

Crucially, convergence is not the desired form of stability in the present task. Unlike fixed-point reasoning, where an input-dependent attractor may represent a completed solution [15], [16], every Rule 30 repeat has a new, generally non-stationary target. A recurrent trajectory is therefore successful only while its decoded states continue to implement valid automaton transitions. The observed convergence is non-benign precisely because it destroys this transition consistency. BUT’s extrapolation beyond its supervised depth is consequently meaningful, but finite: it learns a transition that remains reusable over an extended range without learning one that can be composed indefinitely. More generally, short-horizon transition accuracy does not establish the long-run stability of an iterated learned transition [20].

Table 3. Representative eventual failure of an R_4 BUT. Results are from run 128 (model seed 2) on the final-test split, using the unconstrained-extrapolation checkpoint at training step 3,500, selected using validation performance at 17:17. Diagonal accuracy compares the decoded state D_r with the exact Rule 30 state $X^{(r)}$. Hidden-state cosine compares consecutive recurrent states, and decoded-recurrence accuracy compares D_r with one exact Rule 30 update of D_{r-1} .

Repeat r	Diagonal cell accuracy	Predicted ones	$\cos(H^{(r-1)}, H^{(r)})$	Decoded-recurrence accuracy
5	1.0000	50.02%	0.5415	1.0000
15	0.9584	50.25%	0.7656	0.9805
28	0.4999	64.29%	0.9827	0.5511
36	0.4976	77.41%	0.9902	0.3338
40	0.5006	95.89%	0.9959	0.1007

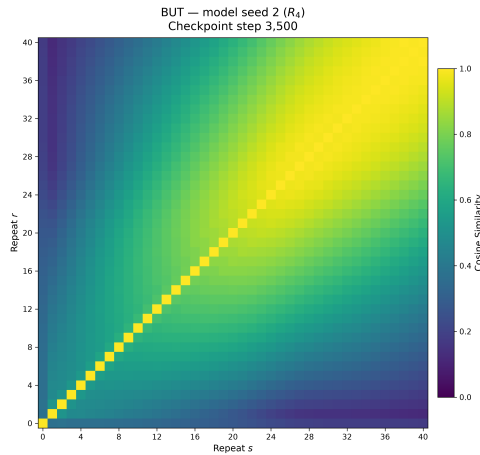


Figure 1. Pairwise cosine similarity between complete recurrent hidden states for a representative R_4 BUT run. Similarity increases among late-depth states near the depth at which prediction accuracy collapses, consistent with convergence of the recurrent dynamics.

3.2.3 Diagnosing CoTFormer’s depth-extrapolation failure

Initial attention diagnostics The R_4 and R_6 CoTFormer runs often fell to chance too abruptly to study intermediate stages of failure. We therefore supervised depths 1:1–12:12 using 500,000 Rule 30 rows and

24,000 optimiser steps. In the strongest run, final-test cell accuracy fell from 0.9993 at repeat 15 to 0.5049 at repeat 20. This extended protocol exposed a gradual failure regime for diagnosis.

In a manual-attention reproduction, the model attended mainly to recent and earliest cached repeats before collapse: at repeat 16, the newest three blocks received 75.04% of attention mass and the earliest three received 16.24%. As accuracy deteriorated, normalised token entropy rose from 0.349 at repeat 12 to 0.936 at repeat 40, while mean maximum attention probability fell from 0.432 to 0.0027. These observations associate failure with diffuse attention but do not establish its cause.

Inference time cache masking We then masked older cache blocks in a frozen checkpoint, retaining either the current and eleven preceding repeats (recent-12) or the current and three preceding repeats (recent-4). Neither restriction rescued extrapolation. At repeat 17, cell accuracy was 0.7526 with the full cache, 0.5369 with recent-12, and 0.4906 with recent-4. The checkpoint therefore depended on information removed by these masks, although the intervention alone cannot show which older blocks were useful.

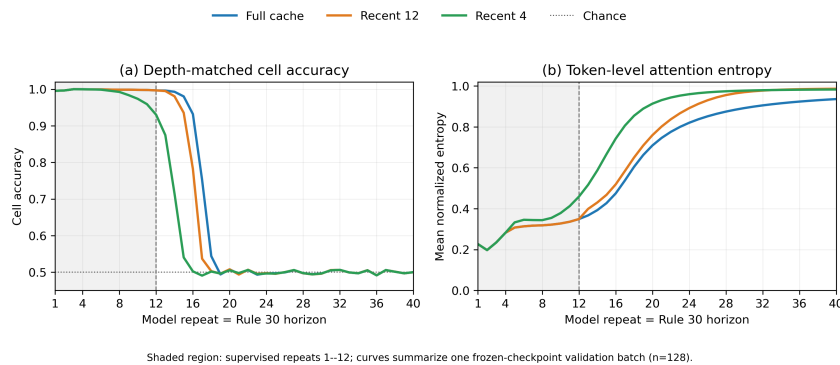


Figure 2. Accuracy and normalised attention entropy under full-cache and recent-only masks. Entropy is normalised by the number of visible key tokens and averaged over examples, heads, and query positions.

Diffuse attention persisted even when recent-4 fixed the visible support at 256 tokens (Figure 2). Pre-softmax logit variation and top-two gaps also shrank with depth. Thus neither a growing softmax denominator nor exploding logits alone explains the failure. These controls motivated the trained cache-policy and state-cache interventions below.

Training time interventions on the cache Because these interventions are applied only at inference time, they expose the frozen model to attention patterns and recurrent states that may differ from those encountered during training. Any resulting change in accuracy may therefore reflect either the intended effect of the intervention or a more general failure under distribution shift. Consequently, these experiments isolate the causal effect of modifying the inference-time computation, but do not establish that the modified computation would remain harmful if the model had been trained under the same constraint.

To address this confound, we trained CoTFormer models with different cache policies. The same masking mechanism used for the inference-time interventions was active throughout training, thereby bounding the number of earlier repeat blocks visible to attention. All models were supervised on the matched pairs 1:1–12:12. Table 4 reports the final-test diagonal accuracy at the training boundary and at three representative extrapolation depths. Each checkpoint was selected using the unconstrained extrapolation criterion. Runs are averaged over model seeds while the data seed and all other scientific configuration fields remain fixed.

The recent-1 policy was the clear exception: the three-seed sweep retained near-perfect accuracy through depth 20. This served as a sanity check as recent 1 becomes functionally equivalent to the Block Universal Transformer. For cache windows two through eight, reducing the window did not consistently

Table 4. Depth-matched final-test cell accuracy for CoTFormer models trained with different repeat-cache policies. Values are mean $\pm$ sample standard deviation across model seeds. Depth 12 is the maximum supervised depth, depth 15 represents early extrapolation, depth 17 lies in the principal failure transition, and depth 20 tests the long-horizon regime. Each cache-policy row contains one observation per unique model seed from the main sweep; earlier pilots that reused those seeds are not counted as additional independent runs.

Training cache	Seeds	12:12	15:15	17:17	20:20
Full	2	0.9974 $\pm$ 0.0009	0.9721 $\pm$ 0.0106	0.6563 $\pm$ 0.1341	0.5008 $\pm$ 0.0001
Recent 1	3	0.9999 $\pm$ 0.0002	0.9995 $\pm$ 0.0006	0.9988 $\pm$ 0.0014	0.9939 $\pm$ 0.0055
Recent 2	3	0.9986 $\pm$ 0.0010	0.9778 $\pm$ 0.0171	0.6236 $\pm$ 0.1009	0.4997 $\pm$ 0.0010
Recent 3	3	0.8325 $\pm$ 0.2872	0.8223 $\pm$ 0.2791	0.6237 $\pm$ 0.1885	0.4995 $\pm$ 0.0014
Recent 4	3	0.9973 $\pm$ 0.0040	0.9819 $\pm$ 0.0168	0.7594 $\pm$ 0.1564	0.5008 $\pm$ <0.0001
Recent 5	3	0.9922 $\pm$ 0.0121	0.9667 $\pm$ 0.0415	0.7033 $\pm$ 0.2549	0.4999 $\pm$ 0.0012
Recent 6	3	0.8315 $\pm$ 0.2875	0.8187 $\pm$ 0.2750	0.5629 $\pm$ 0.1025	0.5004 $\pm$ 0.0006
Recent 7	3	0.9552 $\pm$ 0.0699	0.8959 $\pm$ 0.1432	0.6474 $\pm$ 0.1705	0.4999 $\pm$ 0.0008
Recent 8	3	0.8323 $\pm$ 0.2882	0.8256 $\pm$ 0.2827	0.6055 $\pm$ 0.1069	0.4999 $\pm$ 0.0008

delay failure. Their ordering changed across extrapolation depths, and all reached chance accuracy by depth 20. Several policies also exhibited substantial inter-seed variance at the supervised boundary itself, most notably recent-3, recent-6, and recent-8. The results therefore provide no evidence of a monotonic relationship between bounded cache size and extrapolation distance. They instead suggest that removing cross-repeat attention entirely places the recent-1 model in a qualitatively different computational regime.

Inference time interventions on the recent-k CoTFormer models We conduct 3 different interventions to the CoTFormer trajectories:

Clean baseline. In this intervention, we replace the CoTFormer’s hidden state with the canonical, correct cellular automaton representation. This is done by embedding the representation via the token embedding matrix, adding positional encoding and replacing the hidden state with this representation. We do this once at the intervention depth and let the model run freely after that.

Free reset. We remove all the representations in the CoT cache once during the intervention, and let the model accumulate K and V pairs normally with its trained policy after that.

Clean reset. We do both interventions together.

After the interventions we measure cell accuracy, exact sequence accuracy and the similarity of the hidden states at every subsequent repeat. In these interventions, we notice that the results we get depend highly on the depth at which we intervene. If intervention is done before, or close to the accuracy collapse, generally speaking, clean baseline does not improve, and in some cases may even degrade accuracy. In one case free reset before the collapse even improves the result.

We see that when done around collapsing depths, clean baseline improves the results for a few iterations, delaying the collapse. However when the clean reset intervention is done (resetting the cache and adding a clean hidden state together) we see that extrapolation horizon is essentially restored, and the model is able to extrapolate to its maximum horizon (the most it was ever able to extrapolate).

Factorial state-cache intervention design Together with an unmodified control, the three active manipulations above form a 2×2 factorial intervention. Let t denote the intervention depth. All four conditions use the same examples, checkpoint, free-running trajectory through depth t , and maximum rollout depth of 40. Clearing is a one-time operation. From repeat $t + 1$ onward, every branch again accumulates keys and values normally under the cache policy used to train that model.

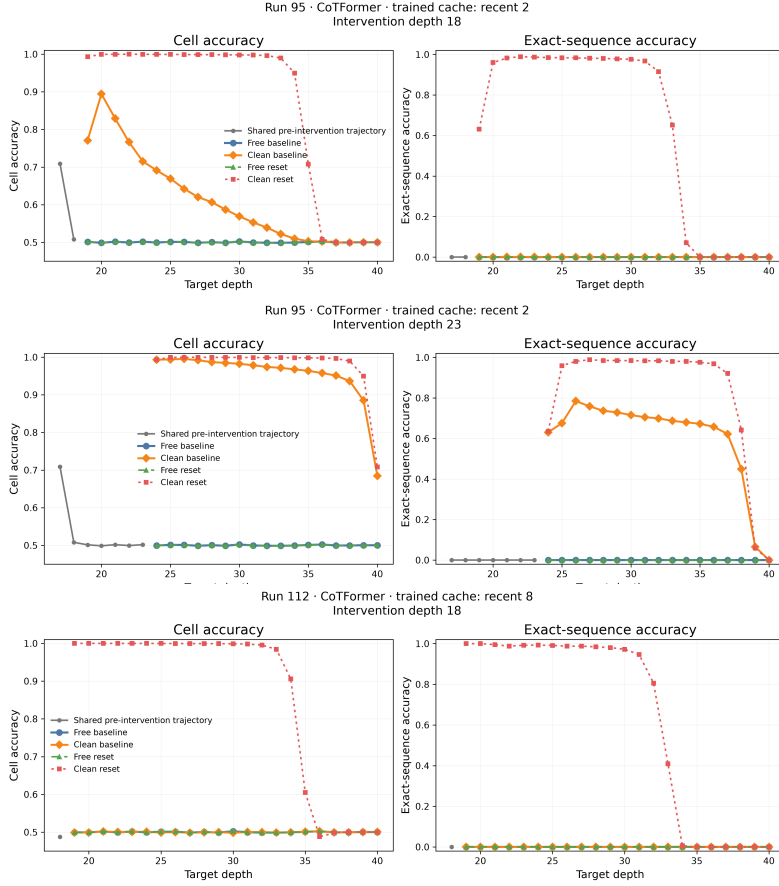


Figure 3. Factorial interventions for recent-2 run 95 at depths 18 and 23 (upper two plots) and recent-8 run 112 at depth 18 (lower plot). Grey points show the shared free-running trajectory and coloured curves the four post-intervention conditions. Clean-state injection rescues the recent-2 model, whereas recent-8 requires both state correction and cache reset.

The first post-intervention prediction is decoded at depth $t + 1$, and cell and exact-sequence accuracy are then measured at every target depth through 40 on the same 4,096-example test split. Thus CB–FB estimates the effect of correcting the current state while retaining its history, FR–FB estimates the effect of removing history without correcting the current state, and CR–CB estimates the additional effect of removing history once the current state is clean. These are paired differences because the examples and the complete pre-intervention computation are shared exactly between conditions.

Figure 3 shows the clearest contrast. Run 95 was trained with a recent-2 cache and had already fallen to chance before the intervention at $t = 23$. A clean state alone raised cell accuracy from 0.4995 to 0.9933 at depth 24 and retained 0.9636 cell accuracy at depth 35; clean reset reached 0.9982 at depth 35. Exact-sequence accuracy at depth 35 was respectively 0, 0.6724, 0, and 0.9761 for FB, CB, FR, and CR. The correction was not permanent: by depth 40 exact-sequence accuracy had returned to zero even under CR. Run 112, trained with a recent-8 cache, behaved differently. At $t = 18$, CB and FR remained at chance, whereas CR reached perfect cell and exact-sequence accuracy at depth 19 and retained 0.9988 cell accuracy and 0.9722 exact-sequence accuracy at depth 30. Its eventual collapse by depth 40 shows that the joint intervention extends a stable trajectory rather than making the transition indefinitely stable.

Dependence on the trained cache window To summarise the larger intervention grid without selecting a single target depth, define a five-repeat post-intervention effect for condition c as the mean cell-accuracy

difference over the first five decoded targets. In particular,

$$\Delta_{\text{state}}(t) = \frac{1}{5} \sum_{j=1}^5 [A_{\text{CB}}(t, t+j) - A_{\text{FB}}(t, t+j)], \quad (1)$$

$$\Delta_{\text{reset|clean}}(t) = \frac{1}{5} \sum_{j=1}^5 [A_{\text{CR}}(t, t+j) - A_{\text{CB}}(t, t+j)], \quad (2)$$

$$\Delta_{\text{reset|free}}(t) = \frac{1}{5} \sum_{j=1}^5 [A_{\text{FR}}(t, t+j) - A_{\text{FB}}(t, t+j)]. \quad (3)$$

For each run, these quantities are first averaged over its tested intervention depths. Table 5 then reports the mean and sample standard deviation across independently trained runs, so a run with more tested intervention depths does not receive greater weight.

Table 5. Five-repeat cell-accuracy effects, averaged within each run and then across runs (mean $\pm$ sample standard deviation). “Interventions” counts tested source depths. Positive Δ_{state} measures the benefit of clean-state injection with the old cache retained; positive $\Delta_{\text{reset|clean}}$ measures the additional benefit of clearing that cache.

Training cache	Runs	Interventions	Δ_{state}	$\Delta_{\text{reset clean}}$	$\Delta_{\text{reset free}}$
Recent 2	3	16	0.182 ± 0.162	0.316 ± 0.166	0.001 ± 0.001
Recent 3	2	10	0.083 ± 0.037	0.414 ± 0.032	0.001 ± 0.001
Recent 4	3	9	0.037 ± 0.057	0.462 ± 0.059	0.000 ± 0.000
Recent 5	3	10	0.017 ± 0.031	0.482 ± 0.031	0.000 ± 0.000
Recent 8	2	7	-0.009 ± 0.014	0.501 ± 0.001	0.004 ± 0.005
Full	2	6	0.003 ± 0.003	0.495 ± 0.003	0.000 ± 0.000

The state-only effect decreases as more recurrent history is retained. It is largest for recent-2, remains positive but smaller for recent-3 and recent-4, and is close to zero for recent-5, recent-8, and full-cache models. The conditional cache-reset effect shows the complementary pattern: once a clean state is supplied, clearing the cache becomes increasingly important as the trained window grows. Resetting the cache without correcting the state has approximately zero mean effect for every policy. The failure is therefore not explained by either a bad current state or a bad cache in isolation. The factorial interaction indicates that a corrected current state can be pulled back toward failure by retained recurrent history, particularly when that history spans many repeats; an empty cache is useful only when paired with a state from which correct computation can resume.

This supports a narrower version of the cache-accelerated-degradation hypothesis. Larger retained histories make an already-corrected state less recoverable, while a sufficiently short history sometimes allows a single state correction to persist. The experiment does not by itself establish that the cache initiated the original collapse, identify which cached token or attention head is responsible, or show a sharp causal threshold at a window of four. Intervention depths were selected around each run’s observed failure region and are not balanced across policies; the table is therefore a descriptive summary rather than an inferential test of monotonicity. Run 113 is excluded from recent-8 because its selected checkpoint was already at chance within the supervised range.

4 Delayed Cellular-Automaton Recall: A Test of Memory

The preceding experiments ask whether recurrent computation continues to implement the Rule 30 transition at depths beyond those used for supervision. They do not directly test the motivation that initially led us to study CoTFormer: whether an intermediate latent state can be retained and selected after subsequent computation has taken place. We therefore extend Rule 30 into a delayed-recall benchmark.

The task retains the evolving, full-sequence state of the cellular automaton, but changes the final objective from predicting the latest state to recalling one specified state from the trajectory.

4.1 Benchmark definition

Let $X^{(0)} \in \{0, 1\}^N$ be an initial row and let $X^{(r)} = f_{30}^r(X^{(0)})$ denote the state after r exact Rule 30 updates. An example is executed for an evolution horizon H , producing the trajectory

$$X^{(0)}, X^{(1)}, \dots, X^{(H)}. \quad (4)$$

After the H evolution repeats, the model receives a query for a target repeat $q \in \{1, \dots, H\}$. The query is represented by its relative age

$$a = H - q, \quad (5)$$

and the required output is the complete earlier row $X^{(q)}$. Thus $a = 0$ requests the current state, whereas larger ages require information from progressively earlier in the trajectory. The input tokens are not replaced by a clean intermediate row during recall: the model receives $X^{(0)}$ once, performs all H evolution repeats, and must then answer through its recurrent state. Moreover, the recall phase occurs after the complete forward trajectory irrespective of q . The model therefore cannot solve an early-state query merely by stopping its evolution at the requested depth.

This construction turns intermediate-state reuse into an explicit supervised task. Additional applications of the forward Rule 30 transition would move away from, rather than towards, a requested past state. A BUT must consequently preserve sufficient trajectory information in its current hidden sequence and learn to reconstruct the requested state from that bounded representation. A CoTFormer has a seemingly more direct route: its attention cache contains keys and values produced at every evolution repeat, so a recall pass can in principle select the block associated with q . The benchmark tests whether that architectural access is actually learned and used as addressable memory.

Rule 30 is well suited to a delayed-recall benchmark precisely because its forward map is lossy. The global update on a ring of N cells is not injective: the all-zeros and all-ones configurations are both mapped to all-zeros, so at least two distinct predecessors collapse onto the same successor at every step. Information present at repeat q is therefore not in general recoverable from the state at repeat H by any amount of further computation, and a model cannot answer a recall query by inverting the dynamics. Retaining the requested state, or features sufficient to reconstruct it, is the only available route.

The task is also close to worst case with respect to compressibility. Rule 30 is a Class III rule whose space-time evolution is empirically incompressible: compression-based classification places it in the cluster whose compressed evolution lengths asymptotically approach their uncompressed lengths [21], and Wolfram’s study of the rule as a pseudo-random sequence generator found its centre column to pass standard statistical randomness tests [22]. Consequently an earlier row shares little exploitable structure with the current one, and a model cannot substitute a cheap summary statistic for genuine storage. We selected Rule 30 for this reason: it converts delayed recall into an unambiguous test of whether a bounded recurrent state can carry information forward, rather than a test of how well a model can compress or re-derive a predictable trajectory.

This contrasts with a reversible automaton, such as a second-order construction in which each update is invertible; there a sufficiently expressive model could in principle recover an earlier state by running the dynamics backwards, confounding memory with recomputation. We initially used second order automata to verify the forward-backward controller works as intended. Controller design detailed in the next section.

4.2 Model and controller design

Both models use the same binary input and output vocabulary, bidirectional attention, rotary positions, and shared Transformer block. BUT carries only the latest hidden sequence; CoTFormer also retains keys and values from earlier repeats, which remain available during recall.

A learned controller specifies the operation d (forward evolution or recall) and relative age a at every sequence position:

$$c(d, a) = e_{\text{dir}}(d) + e_{\text{age}}(a). \quad (6)$$

At forward repeat r , $a = H - r + 1$ counts down to the end of evolution; during recall, $a = H - q$ identifies the requested earlier repeat. Let Z be the current hidden sequence and G_θ one application of the shared block. We use the subtractive update

$$Z' = G_\theta(Z + c(d, a)) - c(d, a). \quad (7)$$

Subtracting the controller prevents its direct residual contribution from accumulating while allowing the block to respond to it. A pilot comparison favoured this update over leaving the controller in the residual stream, so we fixed it for the main experiments.

After all H evolution repeats, the same block runs K times with the recall controller; only its final output is decoded. We test one and two recall applications, using $K = 2$ in the main sweep. A first application could select information from the requested cached repeat, while a second could transform it for prediction. Testing both asks whether extra computation improves recall. BUT and CoTFormer receive the same number of applications; only CoTFormer has direct access to the cross-repeat cache.

4.3 Supervision and training protocol

The model predicts the Rule 30 row after every evolution repeat and the requested earlier row after the final recall application. Let ℓ_r be mean cellwise cross-entropy at evolution repeat r , and let $\mathcal{L}_{\text{query}}$ be the same loss for the recalled row. We optimise

$$\mathcal{L} = \frac{H^{-1} \sum_{r=1}^H \ell_r + 2\mathcal{L}_{\text{query}}}{3}.$$

Thus evolution depths receive equal aggregate weight, the recall response receives twice that weight, and only the final recall application is supervised. Intermediate evolution predictions are decoded for the loss, but clean intermediate rows are never fed back into the model.

Training cycles through horizons $H = 1, \dots, H_{\text{max}}$, one horizon per optimiser step. Within each horizon it also cycles through every requested repeat $q = 1, \dots, H$, ensuring uniform query coverage. The main comparison uses $H_{\text{max}} \in \{6, 12, 20, 30\}$ and three model-initialisation seeds per condition. Models have one shared Transformer block, width 64, four heads, bidirectional attention, and no dropout. We use AdamW (learning rate 10^{-3} , weight decay 0.15), gradient clipping at 0.9, and batch size 128. Runs through horizon 20 train for 5,000 steps; horizon-30 runs train for 10,000. Initial rows are 64 independent Bernoulli(0.5) bits, with 100,000 training rows and separate validation and test sets of 4,096 each. Data and shuffle seeds are fixed across model seeds.

4.4 Recall evaluation and checkpoint selection

We evaluate every query $q = 1, \dots, H$ at each horizon. Checkpoints maximise validation recall cell accuracy averaged first over queries within each horizon and then equally over horizons; worst-query accuracy, exact-row accuracy, forward accuracy, and loss break ties. Final-test results use the selected

checkpoints.

We compare each recalled row with both the true Rule 30 state and the model’s own decoded state at the requested repeat. Agreement with the latter (internal recall accuracy) can be high even when the forward trajectory is wrong or stalled. We therefore also measure agreement on cells where the requested and current decoded states differ, and rank the requested state against every decoded evolution state. On those changed cells, accuracy above 50% favours the requested state over the current one; a *unique-best* match requires a strictly higher similarity than every other candidate. We interpret these diagnostics alongside ground-truth accuracy.

4.5 Delayed-recall results

We first consider a single recall application, before asking whether a second application of the shared block improves retrieval. Figure 4 summarises ground-truth cell accuracy over every nontrivial delayed query. A query with $q = H$ is excluded because it requests the current state and does not require delayed recall. Queries are first averaged within each horizon and the horizon means are then weighted equally, so longer trajectories do not dominate the aggregate merely because they contain more possible queries.

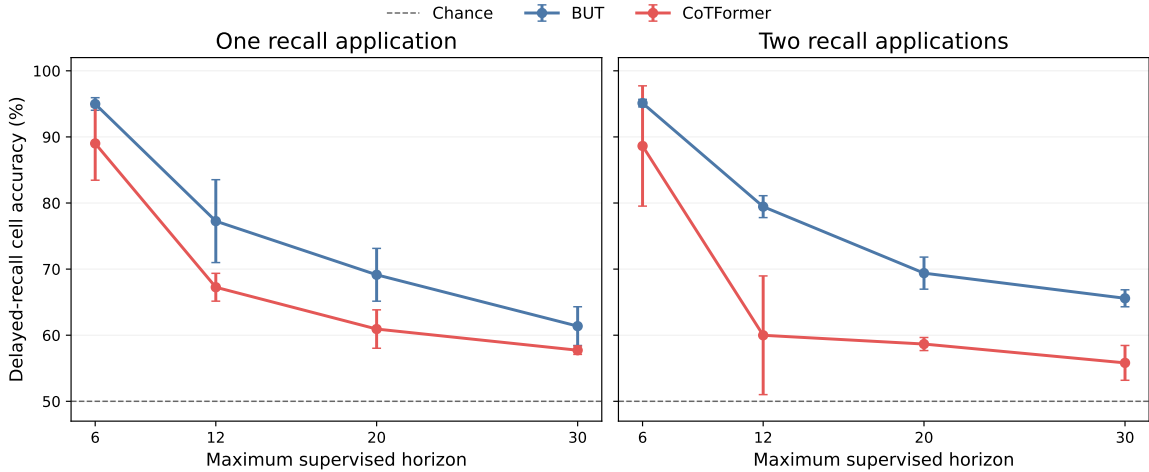


Figure 4. Pair-balanced nontrivial delayed-recall accuracy on the final test split. Points show means over three model-initialisation seeds and error bars show one sample standard deviation. The left and right panels use one and two recall applications respectively. Each point averages the query means for all supervised horizons up to the indicated maximum; the dashed line marks 50% cell accuracy.

With one recall application, BUT achieved higher mean delayed-recall accuracy at every matched maximum horizon (Figure 4). Its advantage over CoTFormer ranged from 3.65 to 10.00 percentage points across the four horizons. These comparisons use three seeds and describe the consistency and scale of the observed difference; the complete means and seed variation are shown in the figure rather than enumerated in the text.

The ground-truth difference does not by itself isolate memory retrieval, because the architectures also differed in how long they could accurately execute Rule 30. The $H_{\max} = 12$ condition provides a representative diagnosis: it contains eleven nontrivial recall ages, both architectures learn the beginning of the trajectory, and their forward accuracies separate only with increasing depth. By repeat 12, BUT retained 87.1% forward accuracy, whereas CoTFormer was at approximately chance. We therefore interpret ground-truth recall together with the internal and selection-sensitive diagnostics in Figure 5.

With one recall application at evaluation horizon 12, raw internal accuracy was nearly identical for BUT and CoTFormer, at 64.8% and 64.1%. Restricting the comparison to cells that distinguish the requested decoded state from the current one reduced CoTFormer to 21.0%, while BUT retained 63.5%.

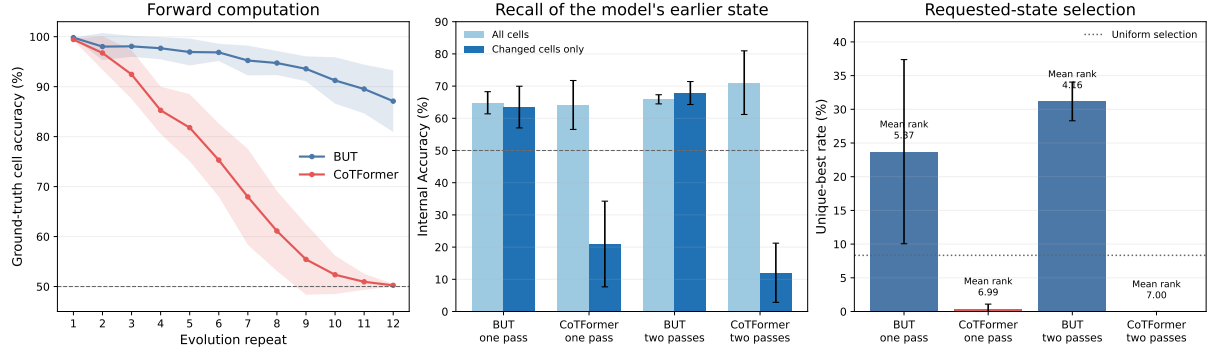


Figure 5. Representative diagnostics for models trained through horizon 12. Left: same-checkpoint Rule 30 accuracy across the forward trajectory for the one-application runs. Centre: raw internal recall accuracy and internal accuracy restricted to cells on which the requested and current decoded states differ, evaluated at horizon 12. Right: the proportion of recalled outputs for which the requested evolution repeat is the unique closest logit-space match. The dotted reference is the 1/12 rate from uniform selection among twelve candidates; annotations report the mean rank of the requested repeat, for which lower is better. Bars and curves show means over three seeds and error bars or bands show one sample standard deviation.

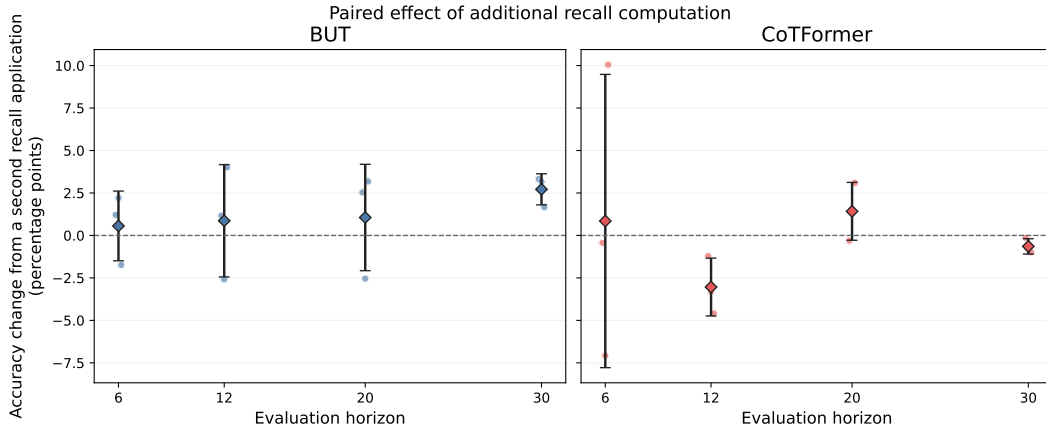


Figure 6. Paired effect of training and evaluating with two rather than one recall application at the indicated evaluation horizon. Small points are the three paired model-seed differences in ground-truth cell accuracy; large points and bars show the paired mean and one sample standard deviation. Positive values favour two applications.

The requested repeat was also the unique closest logit-space match in 23.7% of BUT cases but only 0.43% of CoTFormer cases. Thus CoTFormer’s raw internal agreement did not correspond to reliable selection of the state named by the query.

BUT also achieved higher aggregate ground-truth accuracy at every horizon with two recall applications, with advantages ranging from 6.48 to 19.46 percentage points. The internal diagnostics again reveal more than raw agreement. At horizon 12, CoTFormer had higher raw internal accuracy than BUT, 71.1% against 65.9%, but only 12.0% changed-cell internal accuracy against BUT’s 67.9%. Its requested repeat was almost never the unique closest logit-space match, whereas BUT selected it in 31.2% of cases. Mean ranks and seed variation are reported in Figure 5.

Providing the second recall application produced only modest improvements for BUT and no consistent improvement for CoTFormer (Figure 6). BUT’s paired mean change ranged from +0.56 to +2.71 percentage points. CoTFormer’s changes ranged from −3.04 to +1.42 points and changed sign across horizons. Additional computation therefore did not by itself convert access to the cached trajectory into reliable retrieval.

These results show that the CoTFormer did not learn reliable addressable selection of the requested

cached computation; they do not establish that its cache had no influence on the output. The intervention experiments below examine that causal distinction.

4.5.1 Post-hoc recall-cache interventions

We next asked whether CoTFormer used its recall cache as the intended addressable memory. These are post-hoc validation-set diagnostics rather than part of the principal test-set sweep. The four-head model was the width-64, horizon-20 run-13 checkpoint at step 5000. It used one recall application and was trained with 75% delayed-recall batches. We evaluated ten batches of 128 rows. The head-count ablation comprised six independently trained width-64, single-head models under the same training protocol: three seeds with one recall application and three with two. Each was evaluated on five batches of 128 rows.

Cache notation and interventions The cache is ordered chronologically. We write E_r for the block of 64 keys and values appended while performing forward evolution update r , and R_j for the block appended during recall application j . Consequently, the horizontal axis in Figure 7 runs from E_1 to E_{20} and then to R_1 : the numbering of R_1 identifies the first recall application, not a return to the beginning of the evolution.

We used three interventions. First, *value permutation* replaced the values in the nominally requested evolution block with values from different examples in the same batch, while leaving its keys unchanged. This tests whether the information stored in that block affects the answer. Second, *target masking* prevented the recall computation from attending to the requested evolution block. This tests whether that block is necessary. Third, *target only* hid every other evolution block while leaving the requested block and the recall-phase blocks visible. This tests whether the requested historical block is sufficient given the remaining recall computation. Every condition used the same validation examples in the same order, and all reported changes are relative to the unintervened manual attention baseline.

The block index describes when K/V were written, rather than identifying a clean cellular-automaton state. In particular, E_q is written from the input to forward update q , before that update produces the completed hidden state Z_q ; Z_q first becomes the recurrent input used to write E_{q+1} . The model could learn this fixed offset, but the interventions below directly test the nominal controller-indexed block E_q . They therefore do not rule out distributed use of other blocks or specific use of E_{q+1} .

The heads did not follow the requested repeat For each recall query, we summed attention probability over the 64 key positions in each cache block. Figure 7 also shows the mean norm of each block’s values and the norm of its weighted value contribution before the heads are mixed by the output projection.

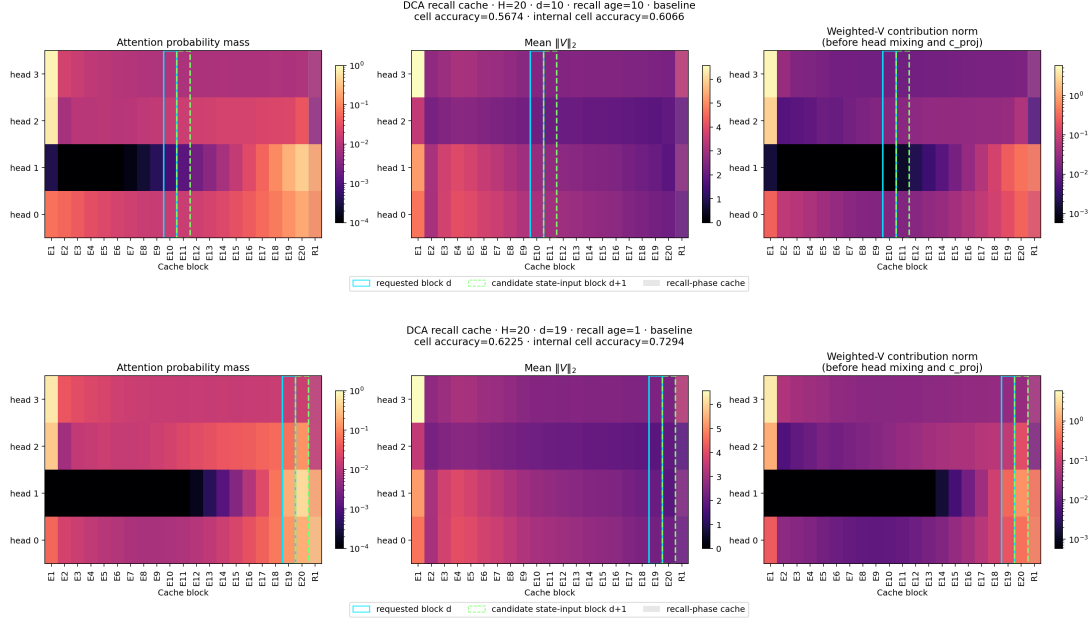


Figure 7. Recall-cache diagnostics for the four-head run-13 checkpoint at horizon 20. The upper plot requests evolution repeat 10 and the lower plot repeat 19. Within each plot, the panels show attention probability, cached-value norm, and weighted-value contribution. Results are averaged over 1,280 validation rows and 64 query positions while retaining the four heads separately. The cyan outline marks E_q ; the green dashed outline marks E_{q+1} . The final column, R_1 , is the cache block written during the recall application.

Changing the requested repeat moved the cyan reference line, but it did not move each head’s preferred cache location with it. The heads instead retained recognizable preferences for particular early or recent blocks. At query 10, the four heads assigned only 0.956% mean mass to E_{10} . It tied for the largest evolution-block mass in just 0.087% of example–head–position observations and had mean rank 11.69 among 20 evolution blocks. At query 19, the mass on E_{19} was larger at 10.584%, but it was the most-attended evolution block in only 0.230% of observations. Its mean rank was 6.87. Attention mass therefore did not track the repeat named by the recall controller.

The requested block was generally neither necessary nor sufficient Table 6 gives absolute cell accuracy for two representative queries. The parenthesized number is the change from normal recall in percentage points. This presentation separates the causal interventions from the descriptive attention measurements above.

Table 6. Four-head run-13 intervention results at horizon 20. Parentheses show the cell-accuracy change from normal recall in percentage points. The last condition retains the requested block as the only visible evolution block, but recall-phase blocks remain visible.

Condition	Query 10	Query 19
Normal recall	56.74%	62.25%
Permute the requested block’s values	56.72% (−0.02)	60.89% (−1.36)
Hide the requested evolution block	56.73% (−0.01)	62.30% (+0.05)
Hide every other evolution block	52.87% (−3.88)	46.65% (−15.60)

At query 10, permuting or hiding the requested block changed accuracy by less than 0.03 points. At query 19, hiding it again caused no loss, despite the larger attention mass in Figure 7. Conversely, keeping only that evolution block reduced query-19 accuracy by 15.60 points and internal cell accuracy by 16.78 points. The requested block was therefore not necessary when the full cache was available and was not

sufficient when the other evolution blocks were removed. The latter result also shows that the cache was not wholly irrelevant: other historical blocks collectively influenced the answer, even though the model did not isolate the nominally requested one.

Using one head did not recover addressing To test whether different heads obscured a query-dependent lookup, we evaluated single-head CoTFormer models across seven non-current horizon-20 queries and three seeds. The requested evolution block received $4.73 \pm 1.04\%$ of attention with one recall application and $3.28 \pm 0.75\%$ with two. It was the most attended block in only $4.66 \pm 8.07\%$ and $2.31 \pm 2.20\%$ of observations, respectively, compared with 5% under uniform selection among 20 evolution blocks. Preferred blocks varied by seed and did not reliably follow the requested repeat.

Interventions supported the same conclusion. Hiding the requested block changed cell accuracy by -0.110 ± 0.183 percentage points with one recall application and -0.011 ± 0.013 with two. Keeping only that evolution block did not reliably help (-1.63 ± 3.06 and $+0.30 \pm 0.31$ points). Similar weak effects at shorter horizons suggest that chance-level horizon-20 accuracy was not the sole explanation. Thus the failure to address the requested block was not an artefact of averaging across heads.

Table 7. Attention to the requested evolution block in the single-head models. Values are means and sample standard deviations over three seeds. Uniform selection among 20 evolution blocks would make the requested block most attended in 5% of observations and give mean rank 10.5.

Recall applications	Mass on requested block	Most attended	Mean rank
1	$4.73 \pm 1.04\%$	$4.66 \pm 8.07\%$	11.80 ± 1.76
2	$3.28 \pm 0.75\%$	$2.31 \pm 2.20\%$	13.37 ± 0.63

Interpretation In principle, CoTFormer can implement the desired lookup. The forward controller supplies an age embedding when every cache block is written, and a recall query could learn to give the matching key a dominant dot product. The training loss, however, supervises the final predicted row rather than the location from which it was retrieved. The model can instead reduce that loss through the latest residual state, controller-conditioned recomputation, diffuse mixtures of cached values, or a fixed preference for particular cache locations. In these experiments, gradient descent consistently found such solutions rather than a query-dependent addressing rule. The one-head ablation shows that this was not merely hidden by averaging several specialized heads.

The result concerns the learned inductive bias rather than formal expressive capacity. It shows that exposing a trajectory through attention does not by itself make that trajectory function as random-access memory. It does not show that attention can never retrieve a past state, nor that every cached block was causally irrelevant.

The delayed-recall models were trained with PyTorch SDPA enabled. Because SDPA does not expose attention probabilities, the subsequent diagnostics used a manual-attention implementation whose predictions were not exactly identical to the fused baseline: agreement was 97.29% for run 13 and ranged from 96.93% to 99.93% across the single-head checks. The use of selected checkpoints and validation data further makes this mechanistic evidence diagnostic rather than an independent confirmatory result. We leave further investigations to future work.

5 Introducing LSTM-UT: A New Gated Recurrent Architecture

The preceding experiments identify a tension between the two original architectures. The BUT provides a stationary, fixed-size recurrent transition and extrapolates more reliably, but has no memory state distinct from its current hidden sequence. CoTFormer retains earlier computation explicitly, but nevertheless

extrapolates less reliably, and retained history can hinder recovery after the current state is corrected. Furthermore, retaining historical key-value blocks does not lead CoTFormer to learn reliable query-dependent addressing. These findings motivated a third architecture intended to combine bounded recurrent computation with a separately persistent memory state: the LSTM Universal Transformer.

5.1 Motivation

A recurrent-depth Transformer must use its propagated representation for two potentially competing purposes: transforming the current state and preserving information that may be needed after further computation. When only the latest residual stream is carried between iterations, newly computed representations can overwrite facts established at earlier depths. Farhan and Chaudhary describe an analogous "concept bottleneck" in continuous latent reasoning and introduce a gated persistent stream across reasoning passes [5]. MeSH independently diagnoses an "information overload" problem in recursive Transformers, in which long-lived and transient information must coexist in one repeatedly transformed state, and addresses it by separating recurrent computation from an explicitly managed memory buffer [23]. Related architectures demonstrate other ways of improving information flow across computational axes. The Feedback Transformer makes high-level representations from earlier tokens available to subsequent computation [24], while Attention Residuals replaces fixed accumulation across network depth with input-dependent selection over preceding layer outputs [25]. Conversely, recent analysis argues that, in ordinary Transformer inference, cached keys and values contain no information independent of the residual representations from which they are deterministically projected [26]. This result concerns autoregressive inference rather than a CoTFormer cache across recurrent depth, but it reinforces the distinction between preserving representations and preserving particular key value projections of them.

Concurrent work by Hegazy et al. introduces the Gated Recurrent Transformer, which uses a single elementwise gate to interpolate between the previous recurrent representation and the output of a shared Transformer core, conditioned on a fixed prelude representation [27]. Both architectures introduce gating across recurrent depth, but they use it differently: the Gated Recurrent Transformer gates the main residual transition, whereas LSTM-UT uses LSTM-style gates to manage a separate, bounded cell state. To the best of our knowledge, this combination of a recurrent-depth Transformer and a distinct cell state with input, forget, and output gating has not previously been studied. In LSTM-UT, the residual stream remains available for transient Transformer computation while the cell state persists across evolution and recall repeats. The experiments that follow test whether this separation supports depth extrapolation and delayed recall more reliably than a single recurrent hidden stream or an expanding history of cached keys and values.

5.2 Architecture and Configurations

The LSTM Universal Transformer augments the recurrent residual stream with a separate persistent memory state. This memory is not itself part of the residual stream and is not supplied directly to either the self-attention or feed-forward sublayer. Instead, the residual stream writes information to the memory through learned gates, and the memory writes information back to the residual stream through a separately gated read. The self-attention and MLP therefore first compute an ordinary Transformer update; the memory mechanism operates only on the block's input and resulting proposal. Unlike CoTFormer's cache, the size of this additional state does not grow with recurrent depth.

For one block application, let z denote the residual-stream representation entering the Transformer block and let

$$\bar{h} = \mathcal{T}_\theta(z) \quad (8)$$

denote the residual-stream representation proposed by its self-attention and feed-forward sublayers. The memory gates do not act directly on these unnormalised representations. A separate learned memory layer normalisation, LN_m , is applied both before and after the Transformer computation:

$$x = \text{LN}_m(z), \quad h = \text{LN}_m(\bar{h}). \quad (9)$$

We refer to x as the *previous hidden state* and h as the *proposed hidden state*. The same LN_m parameters are used for both. This normalisation gives the gates consistently scaled views of the residual stream without placing the persistent memory inside the attention or MLP computation.

The full LSTM-UT configuration computes its forget, write, memory-proposal, and output gates from separate projections of both x and h :

$$f = \sigma(W_{fx}x + W_{fh}h), \quad (10)$$

$$i = \sigma(W_{ix}x + W_{ih}h), \quad (11)$$

$$\tilde{c} = \tanh(W_{cx}x + W_{ch}h), \quad (12)$$

$$c^+ = f \odot c + i \odot \tilde{c}, \quad (13)$$

$$o = \sigma(W_{ox}x + W_{oh}h), \quad (14)$$

$$z^+ = \bar{h} + o \odot \tanh(c^+). \quad (15)$$

Here, c and c^+ are the previous and updated memory-cell states. The first three learned components allow the residual stream to retain, replace, or add information in the memory. The output gate o controls how much of the updated memory is read back into the residual stream. The memory cell c^+ is never passed directly to self-attention or the MLP. Its only immediate effect on the residual stream is the gated additive term $o \odot \tanh(c^+)$. On a later block or recurrent application, z^+ becomes the new residual-stream input, so information read from memory can then influence subsequent Transformer computation.

The final equation is the principal difference from a conventional LSTM. Rather than replacing the hidden representation with $o \odot \tanh(c^+)$, LSTM-UT preserves the complete Transformer proposal $\bar{h}$ and adds a gated read from the memory cell. This keeps the two computational pathways distinct within the current block application: self-attention and the MLP transform the residual stream, while the LSTM-style gates manage persistent memory alongside it. Communication between them occurs only through the gate inputs x and h , which write to the memory, and through $o \odot \tanh(c^+)$, which reads the memory back into the residual stream. The memory cell is initialised to zero for each input and is carried through every evolution and recall application. It has the same sequence length and embedding width as the residual stream and therefore remains fixed in size irrespective of the number of recurrent applications.

At a fixed embedding width, this full configuration is the largest LSTM-UT variant because each of the four gate computations contains one projection of x and one projection of h , giving eight additional square projection matrices. We evaluate three reduced configurations to determine whether all parts of this parameterisation are necessary. The configurations form a 2×2 ablation over the source of the gate inputs and the presence of a learned forget gate:

In the proposed-only variants, every term involving the previous hidden state is removed. For example, the learned-forget proposed-only configuration uses

$$f = \sigma(W_{fh}h), \quad i = \sigma(W_{ih}h), \quad (16)$$

$$\tilde{c} = \tanh(W_{ch}h), \quad o = \sigma(W_{oh}h), \quad (17)$$

while retaining the same cell and residual-stream updates. This ablation is motivated partly by the data flow in a recurrent module containing more than one Transformer block. The complete residual-stream

Table 8. LSTM-UT architectural configurations. "Previous + proposed" means that the remaining gates receive separate projections of both x and h . Removing the forget gate does not remove the write, memory-proposal, or output gates.

Configuration	Forget mechanism	Inputs to learned gates
Full LSTM-UT	Learned f	Previous x and proposed h
Proposed-only	Learned f	Proposed h only
No forget	Exact retention, $f = 1$	Previous x and proposed h
No forget, proposed-only	Exact retention, $f = 1$	Proposed h only

output z^+ of one block becomes the input z of the following block. The next block's previous hidden state is therefore

$$x_{\ell+1} = \text{LN}_{m,\ell+1}(\bar{h}_\ell + o_\ell \odot \tanh(c_\ell^+)). \quad (18)$$

It is consequently derived from the preceding block's Transformer proposal, although it also contains the gated value read from persistent memory and is normalised using the following block's memory layer normalisation. The previous state $x_{\ell+1}$ and the new proposal $h_{\ell+1}$ are therefore distinct, but separate projections of both may provide overlapping information in a deeper recurrent stack. The proposed-only configurations test whether the gates can instead be controlled adequately using only the representation most recently proposed by the current Transformer block.

The present experiments use the 0/1/0 architecture, containing only one recurrent middle Transformer block. They therefore test whether direct access to both the pre-block and post-block representations is useful within one recurrent update; they do not establish that the same projections would be redundant in a multi-block recurrent module. The proposed-only variant should accordingly be interpreted as an architectural and parameter ablation, as well as a design motivated by possible redundancy in deeper models.

The no-forget variants test a different question. A learned forget gate can selectively discard old cell contents, which may be useful when previously stored information becomes irrelevant. Delayed recall, however, may instead benefit from preserving earlier information throughout the forward trajectory. We therefore replace the learned forget gate with exact additive retention:

$$c^+ = c + i \odot \tilde{c}. \quad (19)$$

The write, memory-proposal, and output gates remain learned. Crossing this choice with the proposed-only choice produces the four configurations in Table 8. The comparison separates the observed contribution of learned forgetting from the contribution of conditioning the gates directly on the previous hidden representation. Because removing projections also changes parameter count, the fixed-width configuration comparison is complemented later by separate comparisons against near-parameter-matched BUT models.

5.3 Experimental Scope

We evaluated LSTM-UT by rerunning the two cellular-automaton experiments used for the preceding architectural comparison: ordinary Rule 30 depth extrapolation and delayed cellular-automaton recall. The task definitions, data generation, recurrent 0/1/0 architecture, and training and evaluation protocols were kept aligned with the corresponding BUT experiments so that the effect of replacing the recurrent state mechanism could be assessed under the same experimental conditions. We use BUT as the principal comparator because it was the stronger of the two original architectures in both experiments, outperforming CoTFormer in Rule 30 extrapolation and delayed recall.

We first compare the four LSTM-UT configurations with BUT at the common embedding width of

$d = 64$. Because the additional memory gates give LSTM-UT more trainable parameters at a fixed width, we also ran width-adjusted LSTM-UT and BUT models for near-parameter-matched comparisons. These additional runs test whether any observed advantage persists when model capacity is controlled more closely; they approximately match trainable parameter count, but do not equate floating-point computation, memory traffic, or per-repeat runtime.

5.4 Rule 30 Depth Extrapolation

We first evaluated whether the bounded gated state preserved the ability to apply the learned Rule 30 transition beyond the recurrent depths used during training. To avoid selecting directly for extrapolation, the results below use the in-distribution checkpoint for each run, chosen using performance at the largest supervised depth. The R_4 experiments were trained on 1:1–4:4 and evaluated through 12:12, while the R_6 experiments were trained on 1:1–6:6 and evaluated through 12:12. All models used width $d = 64$, and results are means and sample standard deviations over four model-initialisation seeds for R_4 and three for R_6 . Although the scientific configurations are aligned, the historical baseline runs can differ in evaluation frequency and checkpoint-selection coverage; the cross-architecture numerical comparisons should therefore be read descriptively.

LSTM-UT extrapolated particularly reliably in the R_6 experiment. At depth 12, twice the maximum supervised depth, all four configurations retained greater than 99% mean cell accuracy. The full configuration was strongest, with $99.97 \pm 0.01\%$ cell accuracy and $98.88 \pm 0.30\%$ exact-row accuracy. At the same depth, BUT obtained $98.14 \pm 3.00\%$ cell accuracy and $64.85 \pm 51.64\%$ exact-row accuracy, whereas CoTFormer had fallen to $50.01 \pm 0.05\%$ cell accuracy and zero exact-row accuracy. The LSTM-UT result therefore reflects complete or nearly complete Rule 30 trajectories, rather than high average cell accuracy produced by a small number of residual errors.

The shorter R_4 supervision regime produced a more discriminating comparison between the gate ablations. The learned-forget, proposed-only configuration retained $98.02 \pm 1.25\%$ cell accuracy at depth 9, more than twice its maximum training depth, and $83.29 \pm 9.53\%$ at depth 12. The full learned-forget configuration reached $91.03 \pm 6.67\%$ and $72.73 \pm 14.64\%$ at the same depths. The two no-forget variants deteriorated earlier, showing that extrapolation was not uniform across all LSTM-UT parameterisations. Taken together, the R_4 and R_6 results show that the LSTM-style bounded memory does not prevent recurrent-depth extrapolation and, with sufficient depth supervision, can support an accurate transition at twice the trained recurrent depth.

5.5 Delayed Cellular-Automaton Recall

We evaluated delayed recall using the task, controller, checkpoint selection, and final-test aggregation defined in Section 4. The no-op query $q = H$ is excluded; queries are averaged within each horizon before horizons are weighted equally. All models in this section use the 0/1/0 architecture and width $d = 64$, so these are common-width rather than parameter-matched comparisons. Table 9 gives the complete $H_{\max} = 12$, two-application comparison.

The full LSTM-UT was strongest in this protocol. Removing previous-state gate inputs, learned forgetting, or both reduced cell accuracy by 4.76, 4.19, and 9.14 percentage points, respectively. Even without learned forgetting, LSTM-UT exceeded BUT by 6.75 cell-accuracy points and 33.59 exact-row points. CoTFormer’s gap between internal and ground-truth accuracy also shows why agreement with the model’s own trajectory is insufficient evidence of correct recall.

Table 10 shows that the full LSTM-UT led both baselines at every horizon and recall count, although the best ablation changed with horizon and a second recall application was not uniformly beneficial. These are descriptive results from one shared data seed; historical baselines can also differ in evaluation and

Table 9. Width-64 delayed-recall performance for models trained through horizon 12 with two recall applications. Values are percentages reported as mean $\pm$ sample standard deviation over three model-initialisation seeds on the final-test split, using each run’s validation-selected best delayed recall checkpoint. The macro excludes no-op queries and weights every training horizon equally. “Internal” compares the recalled output with the model’s decoded state at the requested repeat; “GT unique best” is the rate at which the recalled decoded row is uniquely closest in cell agreement to the exact state at the requested repeat among all exact trajectory states.

Architecture	Cell	Internal	Exact row	GT unique best
Full LSTM-UT	90.39 $\pm$ 2.82	90.50 $\pm$ 2.73	48.40 $\pm$ 15.13	83.06 $\pm$ 5.10
Proposed-only	85.63 $\pm$ 4.93	85.95 $\pm$ 4.46	36.88 $\pm$ 19.83	70.76 $\pm$ 12.19
No forget	86.20 $\pm$ 1.48	86.30 $\pm$ 1.43	44.79 $\pm$ 8.07	69.84 $\pm$ 6.95
No forget, proposed-only	81.25 $\pm$ 5.29	81.38 $\pm$ 5.25	25.64 $\pm$ 2.66	60.95 $\pm$ 18.21
BUT	79.45 $\pm$ 1.66	79.79 $\pm$ 1.54	11.20 $\pm$ 8.78	62.05 $\pm$ 2.47
CoTFormer	59.99 $\pm$ 8.98	76.91 $\pm$ 6.30	1.69 $\pm$ 2.93	21.32 $\pm$ 5.64

Table 10. Mean ground-truth cell accuracy (%) across training horizons and recall applications. R1 and R2 denote one and two recall applications. Results average three model-initialisation seeds except the two entries marked *, which have two completed seeds.

Architecture	H6 R1	H12 R1	H20 R1	H6 R2	H12 R2	H20 R2
Full LSTM-UT	98.02	87.93	77.95	97.96	90.39	73.99
Proposed-only	95.61*	84.57	70.28	96.55	85.63	74.37
No forget	95.63	85.12	66.08	96.94	86.20	76.54*
No forget, proposed-only	97.74	81.38	70.70	94.84	81.25	66.25
BUT	94.98	77.26	69.14	95.11	79.45	69.39
CoTFormer	89.01	67.26	60.93	88.63	59.99	58.67

checkpoint coverage. Common width does not control parameter count, and aggregate accuracy does not by itself identify what the cell stores. Parameter matching is therefore considered next.

5.6 Parameter-Matched Comparison

The fixed-width comparison gives LSTM-UT more parameters than BUT. We therefore repeated delayed recall with widths chosen to approximately match trainable parameter count. Table 11 gives the cleanest central comparison: at $H_{\max} = 12$ with two recall applications, the full width-48 LSTM-UT has 6.54% fewer parameters than width-64 BUT, but leads it on all four recall measures.

Table 11. Near-parameter-matched delayed recall at $H_{\max} = 12$ with two recall applications. Accuracy values are percentages reported as mean $\pm$ sample standard deviation over three model-initialisation seeds.

Architecture	Parameters	Cell	Internal	Exact row	GT unique best
Full LSTM-UT, $d = 48$	47,136	88.94 $\pm$ 1.41	89.90 $\pm$ 1.23	18.82 $\pm$ 7.89	84.41 $\pm$ 2.87
BUT, $d = 64$	50,432	79.45 $\pm$ 1.66	79.79 $\pm$ 1.54	11.20 $\pm$ 8.78	62.05 $\pm$ 2.47

Table 12 summarises all six available near-matches. For the two-application horizon-12 protocol, every LSTM-UT configuration outperformed its matched BUT in cell accuracy, by 1.94 to 10.72 percentage points. Across all three protocols, LSTM-UT led in 16 of the 18 pairwise comparisons. The two exceptions were the width-56 no-forget, proposed-only model against width-64 BUT at H12 R1 and H20 R1.

These results rule out additional parameter count as a sufficient explanation for the observed LSTM-UT advantage. They do not establish monotonic scaling: BUT performance itself varied non-monotonically with width. Moreover, matching parameter count does not match computation, memory traffic, or runtime. The evidence is therefore for behavioural parameter efficiency in these protocols, not a population-level or mechanistic explanation.

Table 12. Available near-parameter matches. The parameter gap is $100(P_{\text{LSTM}} - P_{\text{BUT}})/P_{\text{BUT}}$ for H12; H20 gaps differ by at most 0.19 percentage points. Accuracy columns give the LSTM-UT minus BUT difference in ground-truth cell accuracy, in percentage points. R1 and R2 denote one and two recall applications.

Near-matched pair	Parameter gap	H12 R1	H12 R2	H20 R1
Full $d = 48$ / BUT $d = 64$	-6.54%	+9.57	+9.49	+6.27
Proposed-only $d = 56$ / BUT $d = 64$	+1.94%	+8.45	+4.46	+2.53
Proposed-only $d = 64$ / BUT $d = 72$	+5.18%	+5.36	+6.33	+5.82
No forget $d = 64$ / BUT $d = 80$	-4.16%	+8.89	+10.72	+3.63
No forget, proposed-only $d = 56$ / BUT $d = 64$	-4.28%	-0.98	+2.36	-0.70
No forget, proposed-only $d = 64$ / BUT $d = 72$	-1.26%	+1.64	+1.94	+6.24

5.7 Interpretation and Limitations

Taken together, the experiments identify bounded gated state as a promising inductive bias for recurrent-depth computation. LSTM-UT retains the fixed-size recurrent interface of BUT while giving the model a separate state whose contents can be retained, updated, and exposed through learned gates. This separation provides a plausible way to reduce competition between transforming the current residual stream and preserving information for later use. The results are consistent with that interpretation: LSTM-UT supported accurate Rule 30 transitions beyond the supervised recurrent range and produced more reliable delayed recall than either original architecture. They do not, however, establish that this separation is the mechanism responsible for the improvement.

The depth-extrapolation results show that adding gated memory does not prevent a shared transition from being reused at unseen depths. Under the deeper supervision regime, every tested LSTM-UT configuration remained accurate at twice the maximum trained depth. The shorter supervision regime was less uniform and distinguished the gate configurations more clearly: learned-forget variants extrapolated farther than the corresponding no-forget variants, while the proposed-only configuration could outperform the full configuration. The benefit therefore cannot be reduced to the presence of the largest or most heavily parameterised gating mechanism. It depends on the training regime and on how the persistent state is coupled to the Transformer proposal.

Previous recurrent-depth designs have used residual paths between iterations and connections back to the initial loop state [13], [28] to preserve information during repeated computation. LSTM-UT’s cell offers a related but adaptive route: its gates can retain information while the residual stream continues to change. In principle, this could delay the eventual accuracy collapse observed in BUT. The present experiments do not test whether it does so.

Delayed recall provides stronger evidence that the persistent state is useful when information from an earlier recurrent step must remain available after further computation. The full LSTM-UT was the strongest common-width model in the principal delayed-recall comparison and led both original architectures across the evaluated horizons and recall counts. Removing previous-state gate inputs, learned forgetting, or both reduced performance in the central horizon-12 protocol. Nevertheless, no ablation ordering held in every condition, and a second recall application was not uniformly beneficial. The experiments therefore support the complete gated design as an effective default for this benchmark, rather than establishing that every gate is independently necessary or that more recall computation is always useful.

The near-parameter-matched comparisons make additional parameter count an insufficient explanation for the delayed-recall advantage. LSTM-UT generally retained its advantage when its width was reduced to approximately match, and in one central comparison fell below, the parameter count of BUT. Parameter matching does not equalise computation, memory traffic, optimisation difficulty, or runtime, so these results establish behavioural parameter efficiency only within the tested protocols. They do not establish that LSTM-UT is uniformly more computationally efficient.

The internal role of the LSTM cell remains unresolved. Aggregate recall accuracy does not reveal

whether the cell stores compressed snapshots of the trajectory, features from which earlier rows can be reconstructed, a controller-dependent summary, or information that merely improves the residual stream’s own recurrent dynamics. Unlike the CoTFormer analysis, the present study does not intervene directly on the LSTM cell, inspect its gates across depth, or test whether a particular cell state is necessary or sufficient for recall. Targeted cell-state replacement and ablation, gate-activation analysis, and probes for individual trajectory states would be needed to support a mechanistic account.

Finally, the evidence is restricted to deterministic binary cellular automata and their delayed-recall extension. The experiments use a limited number of model-initialisation seeds and a shared data seed, and some historical baselines differ in evaluation frequency and checkpoint coverage. Rule 30 provides a precise test of repeated transition and recall, but it does not reproduce the semantic variability, partial observability, stochastic dynamics, or open-ended outputs of natural-language reasoning and learned world models. The results therefore justify treating bounded gated memory as a strong candidate for those settings, not assuming that its advantage will transfer without direct evaluation.

6 Conclusion

This paper examined how recurrent-depth Transformers balance repeated computation with the preservation of earlier information. It compared a Block Universal Transformer, which carries only its latest hidden sequence; CoTFormer, which augments that sequence with an expanding cache of recurrent keys and values; and LSTM-UT, which introduces a bounded gated cell state. Rule 30 cellular automata isolated recurrent transition stability, while delayed cellular-automaton recall tested whether information from an earlier recurrent step could be recovered after the computation had continued.

The first question was whether a shared transition can remain useful beyond the recurrent depths on which it was supervised. BUT extrapolated more reliably than CoTFormer across the original Rule 30 experiments, demonstrating that a bounded current state can support meaningful depth extrapolation. However, BUT eventually deteriorated under excessive recurrence and approached an inaccurate, highly similar decoded trajectory. Its success was therefore substantial but finite: weight sharing made additional applications possible, but did not make the learned transition indefinitely stable.

The CoTFormer experiments showed that exposing more recurrent history does not by itself produce either stable extrapolation or dependable memory. State–cache interventions found that its depth-dependent failure could not be explained by a corrupted current state or retained history in isolation. Correcting the state could restore computation temporarily, but longer retained histories could draw that corrected trajectory back towards failure; clearing the cache was most useful when paired with a corrected state, strengthening the cache accelerated latent collapse hypothesis for the recent-k models.

Delayed recall tested the intended memory advantage of CoTFormer more directly. Although CoTFormer could attend to representations from every preceding recurrent step, BUT achieved higher ground-truth recall accuracy. CoTFormer’s attention did not reliably follow the requested repeat, and the nominally requested cache block was generally neither necessary nor sufficient for its answer. High agreement with the model’s own decoded trajectory could also coexist with poor agreement with the true cellular-automaton trajectory. The relevant distinction is therefore not simply whether past representations are available, but whether training leads the model to preserve and select them in a form that supports the required computation.

These findings motivated LSTM-UT as a bounded gated alternative. LSTM-UT gave the Transformer a persistent state distinct from the residual stream without allowing the memory footprint to grow with recurrent depth. It produced the strongest overall Rule 30 and delayed-recall results, and its recall advantage largely remained under near-parameter-matched comparisons. The ablations indicate that learned forgetting and access to both the previous and proposed hidden states can be useful, while also showing that no

single gate configuration dominates under every training horizon. Bounded gated memory is therefore supported as an effective inductive bias in these experiments, not as a sufficient or universally optimal solution to recurrent reasoning.

The broader implication is that computational depth and memory architecture must be evaluated separately. A model can apply a shared block repeatedly without preserving the information required later, and it can expose an entire history without learning to address that history reliably. Conversely, a bounded state need not be a disabling bottleneck when its retention and update are explicitly controlled. This distinction is relevant to iterative reasoning and learned dynamics, but the present evidence remains architectural and synthetic.

Future work should test LSTM-UT on natural-language tasks that require delayed reuse of intermediate reasoning, and on learned world models with partial observability, actions, and stochastic transitions. It should also intervene directly on the LSTM cell and its gates to determine what information is stored and how it affects later computation. It would also be useful to test whether the persistent cell state acts as an anchor that delays the eventual collapse observed in BUT. Larger seed sweeps, independent data seeds, compute-matched baselines, and evaluation over longer and more varied trajectories would clarify the reliability and scope of the observed advantage. Until those tests are performed, the strongest conclusion is deliberately narrow: on the cellular-automaton experiments studied here, bounded gated memory provides a more reliable basis for recurrent extrapolation and delayed recall than either an ungated current state or an expanding attention cache.

References

- [1] M. Dehghani, S. Gouws, O. Vinyals, J. Uszkoreit, and L. Kaiser, “Universal transformers,” *arXiv preprint arXiv:1807.03819*, 2018.
- [2] N. Saunshi, N. Dikkala, Z. Li, S. Kumar, and S. J. Reddi, “Reasoning with latent thoughts: On the power of looped transformers,” in *International Conference on Learning Representations*, 2025. [Online]. Available: <https://arxiv.org/abs/2502.17416>
- [3] H. Kohli, S. Parthasarathy, H. Sun, and Y. Yao, “Loop, think, & generalize: Implicit reasoning in recurrent-depth transformers,” *arXiv preprint arXiv:2604.07822*, 2026. [Online]. Available: <https://arxiv.org/abs/2604.07822>
- [4] S. Hao et al., “Training large language models to reason in a continuous latent space,” in *Second Conference on Language Modeling*, 2025. [Online]. Available: <https://openreview.net/forum?id=Itxz7S4Ip3>
- [5] M. Farhan and M. Chaudhary, “Why limit the residual stream to layers and not tokens? persistent memory for continuous latent reasoning,” *arXiv preprint arXiv:2606.07720*, 2026. [Online]. Available: <https://arxiv.org/abs/2606.07720>
- [6] A. Mohtashami, M. Pagliardini, and M. Jaggi, “CoTFormer: A chain of thought driven architecture with budget-adaptive computation cost at inference,” in *International Conference on Learning Representations*, 2025. [Online]. Available: <https://openreview.net/forum?id=7igPXQFupX>
- [7] D. Hafner, T. Lillicrap, J. Ba, and M. Norouzi, “Dream to control: Learning behaviors by latent imagination,” in *International Conference on Learning Representations*, 2020. [Online]. Available: <https://arxiv.org/abs/1912.01603>
- [8] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap, “Mastering diverse control tasks through world models,” *Nature*, vol. 640, pp. 647–653, 2025. DOI: 10.1038/s41586-025-08744-2

- [9] J. Schrittwieser et al., “Mastering atari, go, chess and shogi by planning with a learned model,” *Nature*, vol. 588, pp. 604–609, 2020. doi: 10.1038/s41586-020-03051-4
- [10] N. Hansen, H. Su, and X. Wang, “TD-MPC2: Scalable, robust world models for continuous control,” in *International Conference on Learning Representations*, 2024. [Online]. Available: <https://openreview.net/forum?id=0xh5CstDJU>
- [11] V. Micheli, E. Alonso, and F. Fleuret, “Transformers are sample-efficient world models,” in *International Conference on Learning Representations*, 2023. [Online]. Available: <https://openreview.net/forum?id=vhFu1Acb0xb>
- [12] J. Bruce et al., “Genie: Generative interactive environments,” in *Proceedings of the 41st International Conference on Machine Learning*, vol. 235, PMLR, 2024, pp. 4603–4623. [Online]. Available: <https://proceedings.mlr.press/v235/bruce24a.html>
- [13] L. Yang, K. Lee, R. D. Nowak, and D. Papailiopoulos, “Looped transformers are better at learning learning algorithms,” in *International Conference on Learning Representations*, 2024. [Online]. Available: <https://openreview.net/forum?id=XpVoUnPuYV>
- [14] A. Schwarzschild et al., “Can you learn an algorithm? generalizing from easy to hard problems with recurrent networks,” in *Advances in Neural Information Processing Systems*, vol. 34, 2021. [Online]. Available: <https://openreview.net/forum?id=Tsp2PL7-GQ>
- [15] X.-W. Yang et al., “Stabilizing recurrent dynamics for test-time scalable latent reasoning in looped language models,” in *Proceedings of the 43rd International Conference on Machine Learning*, 2026. [Online]. Available: <https://arxiv.org/abs/2605.26733>
- [16] S. Movahedi et al., “Fixed-point reasoners: Stable and adaptive deep looped transformers,” *arXiv preprint arXiv:2606.18206*, 2026. [Online]. Available: <https://arxiv.org/abs/2606.18206>
- [17] R.-J. Zhu et al., “Scaling latent reasoning via looped language models,” *arXiv preprint arXiv:2510.25741*, 2025. [Online]. Available: <https://arxiv.org/abs/2510.25741>
- [18] H. Zhang, “Chain-of-thought and compressed looped transformers: A memory-budget separation,” *arXiv preprint arXiv:2605.30757*, 2026. [Online]. Available: <https://arxiv.org/abs/2605.30757>
- [19] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, “Language models are unsupervised multitask learners,” OpenAI, Tech. Rep., 2019. [Online]. Available: https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf
- [20] A. Pervez and F. Locatello, “Controlling transient amplification improves long-horizon rollouts,” *arXiv preprint arXiv:2605.08856*, 2026. [Online]. Available: <https://arxiv.org/abs/2605.08856>
- [21] H. Zenil, “Compression-based investigation of the dynamical properties of cellular automata and other systems,” *Complex Systems*, vol. 19, no. 1, pp. 1–28, 2010, arXiv:0910.4042.
- [22] S. Wolfram, “Random sequence generation by cellular automata,” *Advances in Applied Mathematics*, vol. 7, no. 2, pp. 123–169, 1986. doi: 10.1016/0196-8858(86)90028-X
- [23] C. Yu et al., “MeSH: Memory-as-state-highways for recursive transformers,” in *International Conference on Learning Representations*, 2026. [Online]. Available: <https://openreview.net/forum?id=IhTrFvY7p3>

- [24] A. Fan, T. Lavril, E. Grave, A. Joulin, and S. Sukhbaatar, “Addressing some limitations of transformers with feedback memory,” *arXiv preprint arXiv:2002.09402*, 2020. [Online]. Available: <https://arxiv.org/abs/2002.09402>
- [25] Kimi Team, “Attention residuals,” *arXiv preprint arXiv:2603.15031*, 2026. [Online]. Available: <https://arxiv.org/abs/2603.15031>
- [26] K. U. Qasim, J. Zhang, M. K. Shaheen, R. Alharith, and H. Zhang, “The residual stream is all you need: On the redundancy of the KV cache in transformer inference,” *arXiv preprint arXiv:2603.19664*, 2026. [Online]. Available: <https://arxiv.org/abs/2603.19664>
- [27] A. Hegazy, A. Alanwar, and M. Elhoushi, “Gated recurrent transformers: Expressive depth through recurrent modulation,” *arXiv preprint arXiv:2608.15062*, 2026. doi: 10.48550/arXiv.2608.15062 [Online]. Available: <https://arxiv.org/abs/2608.15062>
- [28] J. Geiping et al., “Scaling up test-time compute with latent reasoning: A recurrent depth approach,” in *Advances in Neural Information Processing Systems*, vol. 38, 2025, pp. 41 340–41 391. doi: 10.52202/085713-1380 [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2025/hash/3b01972cf31e6fa0fe29e4b8b5c2a0a1-Abstract-Conference.html